



**UNIVERSIDAD  
DE CUENCA**

## **Documento de diseño y desarrollo**

AgroSmart

### **Docente:**

Miguel Angel Zúñiga, PhD.

### **Estudiantes:**

David Alvarado

Kevin Coraizaca



# UNIVERSIDAD DE CUENCA

|                                                                          |    |
|--------------------------------------------------------------------------|----|
| 1. Introducción                                                          | 6  |
| 1.1. Propósito                                                           | 6  |
| 1.2. Ámbito                                                              | 6  |
| 2. Diseño arquitectónico                                                 | 7  |
| 2.1. Vista de módulos                                                    | 7  |
| 2.2. Vista de componentes                                                | 8  |
| 2.3. Arquitectura física ( Diagrama de despliegue)                       | 11 |
| 3. Diseño Detallado de Módulos (Diagramas de Clases)                     | 14 |
| 3.1. Diagrama de Clases: Módulo de Adquisición y Control (M.A.C.)        | 14 |
| Análisis de Trazabilidad con la ERS (M.A.C.)                             | 15 |
| 3.2. Diagrama de Clases: Módulo de Comunicaciones y Servidor (M.C.S.)    | 16 |
| 3.3. Diagrama de Clases: Módulo de Interfaz (M.I.P.)                     | 18 |
| Análisis de Trazabilidad (M.I.P. / David Alvarado)                       | 19 |
| 4. Diseño de Interfaces Gráficas de Usuario                              | 19 |
| 4.1. Interfaz de Monitoreo de Cultivo (Dashboard)                        | 20 |
| 4.2. Interfaz de Configuración de Automatización (Reglas)                | 22 |
| 4.3. Interfaz de Control Manual de Actuadores                            | 24 |
| 5. Diseño Dinámico (Diagramas de Secuencia Detallados)                   | 26 |
| 5.1. Realización del Caso de Uso: "Monitoreo Remoto de Cultivo"          | 26 |
| 5.2. Realización del Caso de Uso: "Procesamiento de Telemetría y Reglas" | 27 |
| 5.3. Realización del Caso de Uso: "Ciclo de Adquisición en Hardware"     | 28 |
| 6. Diseño de Interfaz con Otros Sistemas                                 | 29 |
| 6.1. Interfaz con Servicio Meteorológico Externo (OpenWeatherMap)        | 30 |
| 6.1.1. Interfaz lógica                                                   | 30 |
| 6.1.2. Interfaz Física y Protocolo                                       | 30 |
| 6.1.3. Datos a Intercambiar                                              | 30 |
| 6.2. Interfaz con Dispositivos de Hardware                               | 31 |
| 6.2.1. Interfaz lógica                                                   | 31 |
| 6.2.2. Especificación de Interfaz Física                                 | 31 |
| 7. Matriz de Trazabilidad                                                | 32 |
| 7.1. Matriz de Trazabilidad: Requerimientos vs. Clases de Software       | 32 |
| 7.2. Matriz de Trazabilidad: Casos de Uso vs. Interfaces Gráficas        | 33 |
| 7.3. Matriz de Trazabilidad: Requerimientos vs. Hardware y Firmware      | 34 |
| 8. Referencias                                                           | 35 |



## 1. Introducción

### 1.1. Propósito

El propósito de este documento de Diseño y Desarrollo (SDD) es describir la arquitectura del software, las interfaces y el diseño detallado del sistema **AgroSmart**. Este documento toma como base los requisitos funcionales y no funcionales descritos en la Especificación de Requisitos de Software (ERS) y los traduce en una solución técnica implementable.

### 1.2. Ámbito

El diseño del sistema **AgroSmart** abarca una solución IoT (Internet of Things) integral para la automatización y monitoreo de cultivos. El alcance técnico del diseño se estructura en cinco módulos funcionales interconectados:

**Módulo de Adquisición y Control (M.A.C.):** Diseño del firmware para el hardware de campo (basado en Keyestudio/Arduino), encargado de la lectura de sensores (Humedad, Temperatura, Luz) y el control físico de actuadores de riego.

**Módulo de Comunicaciones y Servidor IoT (M.C.S.):** Arquitectura del backend que centraliza la lógica de negocio, gestiona la persistencia de datos históricos y orquesta la comunicación entre los dispositivos.

**Módulo de Gestión y Configuración (M.G.C.):** Subsistema lógico encargado de la administración de reglas de negocio, incluyendo el alta de usuarios, definición de zonas de cultivo y la configuración de los umbrales operativos que rigen la automatización.

**Módulo de Interfaz y Presentación (M.I.P.):** Diseño de la aplicación web/móvil que permite a los actores visualizar telemetría en tiempo real y gestionar el sistema.

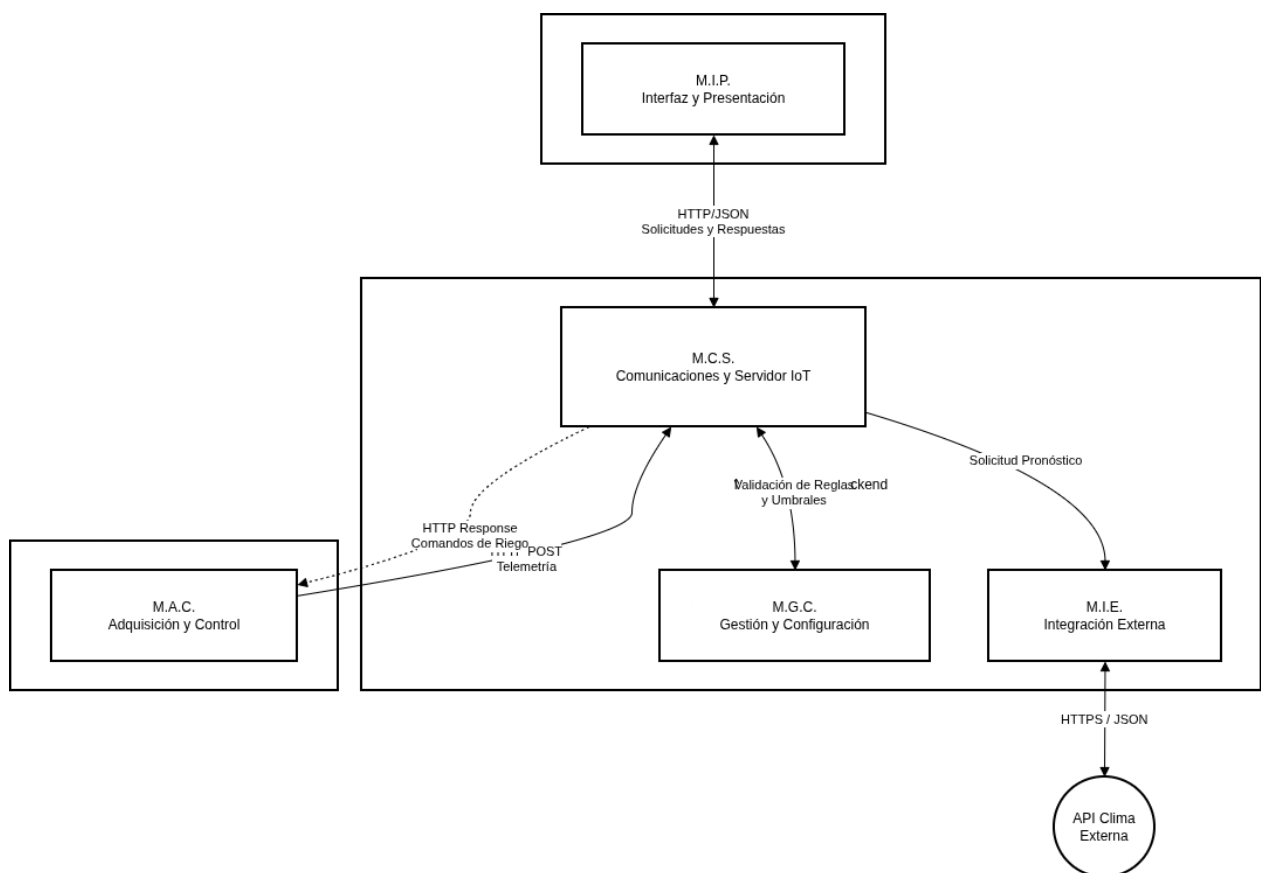
**Módulo de Integración Externa (M.I.E.):** Definición de las interfaces para el consumo de APIs meteorológicas externas, permitiendo la adaptación del sistema a condiciones climáticas.



## 2. Diseño arquitectónico

### 2.1. Vista de módulos

Esta vista presenta la descomposición lógica del sistema AgroSmart, refinando el modelo presentado en la Especificación de Requerimientos. La arquitectura adopta un patrón **Centrado en el Servidor (Server-Centric)**, donde el Módulo de Comunicaciones y Servidor (M.C.S.) actúa como el núcleo de orquestación, desacoplando la lógica de adquisición física (M.A.C.) de la lógica de presentación (M.I.P.).



### Descripción de los Módulos e Interacciones:

**Módulo de Comunicaciones y Servidor IoT (M.C.S.):** Funciona como el bus de integración del sistema. Recibe tramas de datos crudos provenientes del campo, las procesa según las reglas de negocio definidas y gestiona el almacenamiento persistente. Expone una API REST para servir datos al cliente.



# UNIVERSIDAD DE CUENCA

**Módulo de Adquisición y Control (M.A.C.):** Responsable de la interacción física. Opera de forma cíclica enviando telemetría al M.C.S. y ejecutando órdenes de control (riego) recibidas como respuesta del servidor.

**Módulo de Interfaz y Presentación (M.I.P.):** Capa de cliente que consume los servicios del M.C.S. para visualización de datos en tiempo real y reportes históricos. No tiene comunicación directa con el hardware (M.A.C.), garantizando la seguridad del sistema.

**Módulo de Gestión y Configuración (M.G.C.):** Subsistema transversal encargado de definir los parámetros operativos (umbrales de temperatura/humedad). Sus configuraciones son almacenadas por el M.C.S. y determinan el comportamiento automático del sistema.

**Módulo de Integración Externa (M.I.E.):** Actúa como un *Adapter*, encapsulando la complejidad de la comunicación con servicios meteorológicos de terceros y estandarizando los datos entrantes para el uso interno de AgroSmart.



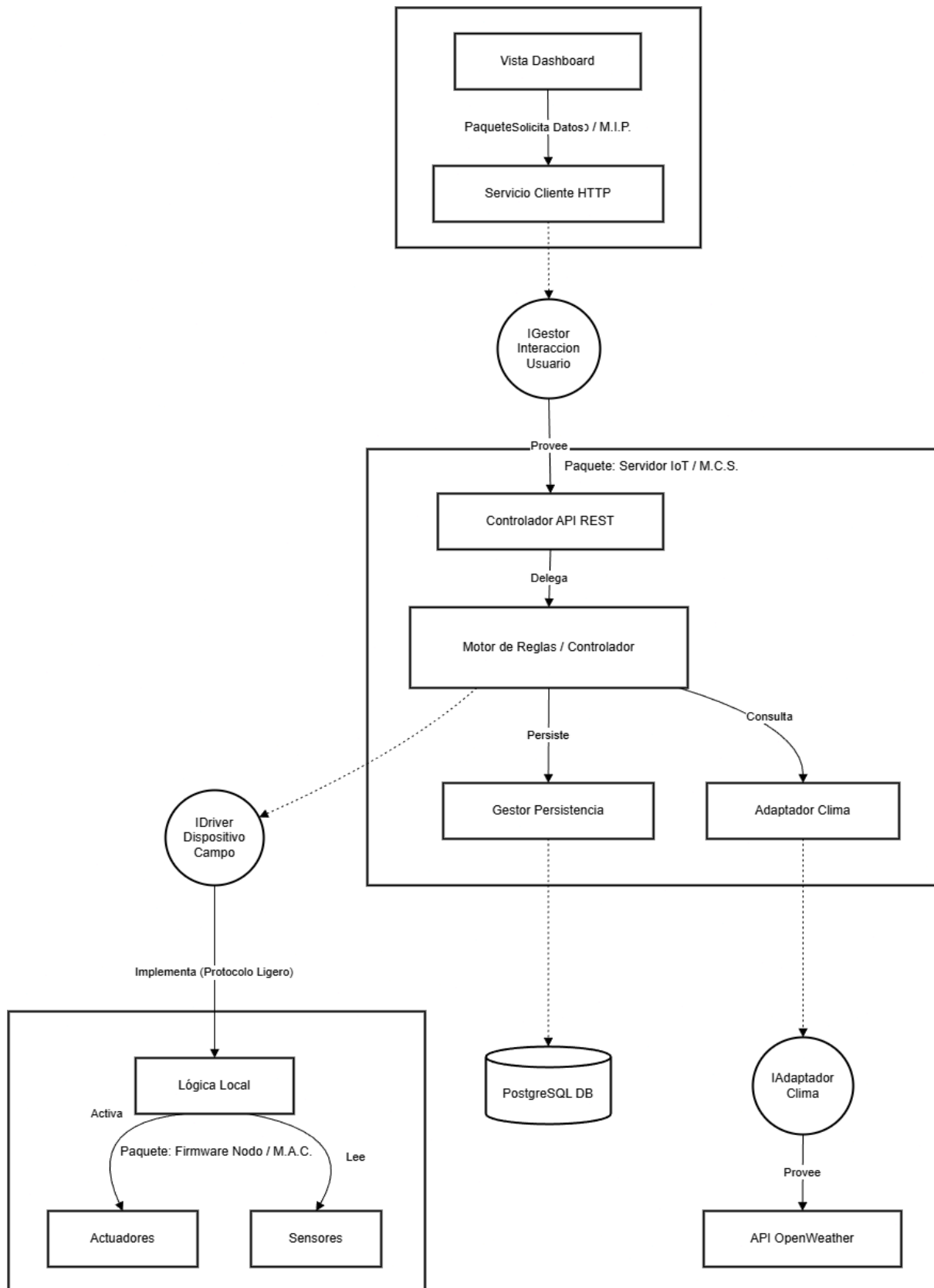
UNIVERSIDAD  
DE CUENCA

## 2.2. Vista de componentes

El siguiente diagrama ilustra la organización de los componentes dentro de los paquetes de despliegue (Hardware, Servidor y Cliente) y sus relaciones de dependencia:



# UNIVERSIDAD DE CUENCA





A continuación se describen los componentes clave y los contratos (interfaces) que garantizan la interoperabilidad:

## Subsistema Servidor (M.C.S., M.G.C., M.I.E.)

- **Controlador API REST:** Componente de frontera que expone la interfaz **I\_AgroAPI**. Es responsable de recibir peticiones HTTP (GET/POST), validar tokens de seguridad y deserializar los objetos JSON entrantes.
- **Motor de Reglas (Lógica M.G.C.):** Núcleo lógico que implementa las reglas de negocio. Recibe la telemetría validada y decide si se debe activar una alerta o un actuador basándose en los umbrales configurados.
- **Gestor de Datos (DAO):** Implementa el patrón *Data Access Object*. Abstrae la complejidad de las consultas SQL a través de la interfaz **I\_Persistencia**, permitiendo que el resto del sistema guarde o recupere datos sin conocer el motor de base de datos específico.
- **Adaptador Clima (M.I.E.):** Componente tipo *Adapter* que traduce las respuestas complejas de la API externa (OpenWeather) al modelo de datos simplificado interno de AgroSmart.

## Subsistema Hardware (M.A.C.)

- **Gestor de Sensores:** Módulo del firmware encargado de interactuar con la **Interfaz GPIO** para leer señales eléctricas de los sensores (DHT11, Higrómetro) y convertirlas a unidades de ingeniería.
- **Cliente WiFi:** Encapsula la pila de protocolos TCP/IP. Es responsable de armar el paquete de telemetría y enviarlo al servidor cumpliendo el contrato de la interfaz **I\_AgroAPI**.

## Subsistema Cliente (M.I.P.)

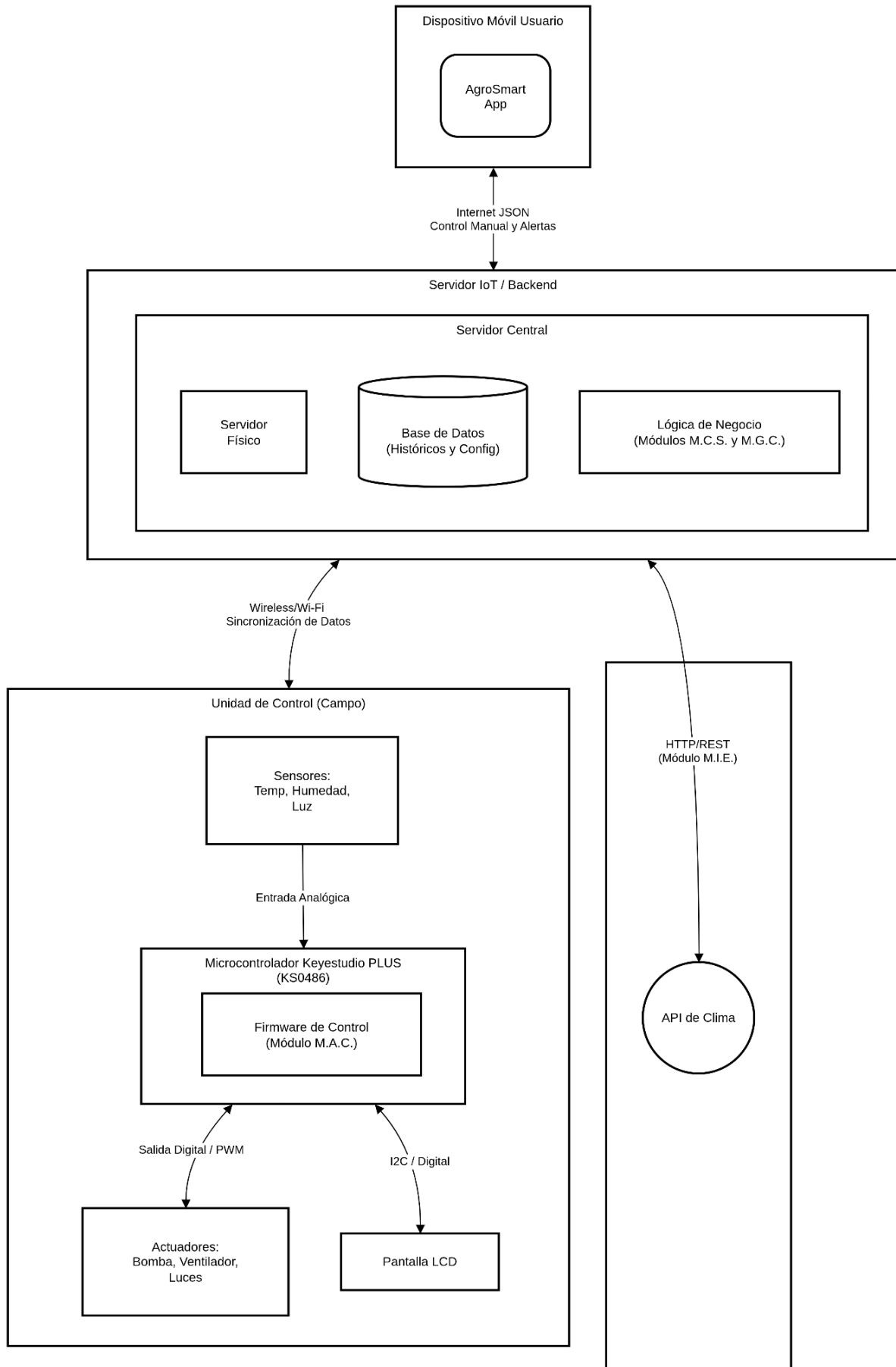
- **Servicio Cliente HTTP:** Componente del frontend (JavaScript/Framework) que gestiona las llamadas asíncronas (AJAX/Fetch) hacia el servidor, manejando tiempos de espera y errores de red.

## 2.3. Arquitectura física (Diagrama de despliegue)

Esta vista describe la topología de hardware del sistema AgroSmart, identificando los nodos físicos de procesamiento, los dispositivos de red y los protocolos de comunicación utilizados para interconectar los entornos de campo, servidor y cliente. El siguiente diagrama ilustra la distribución de los artefactos de software sobre la infraestructura de hardware:



# UNIVERSIDAD DE CUENCA





La arquitectura física se compone de tres niveles jerárquicos de procesamiento:

## Nivel de Campo (Device Tier)

- **Nodo:** Unidad de Control AgroSmart.
- **Hardware:** Se utiliza la placa de desarrollo **Keyestudio PLUS (KS0486)**, basada en el microcontrolador ATmega328P (16MHz, 2KB SRAM). Dado que esta placa carece de conectividad nativa, se integra un **Módulo WiFi (ESP8266/ESP32)** como puente de comunicación.
- **Artefacto:** Ejecuta el **Firmware de Control**, encargado de la adquisición de señales de los sensores y la conmutación de los actuadores (relés) en tiempo real estricto.

## Nivel de Servidor (Application Tier)

- **Nodo:** Servidor de Aplicaciones (Instancia en la Nube o Servidor Local).
- **Hardware Virtual:** CPU vCore, 2GB RAM mínimo para soporte de concurrencia.
- **Artefactos:**
  - *Backend Runtime:* Ejecuta la lógica del M.C.S. y expone la API REST.
  - *Base de Datos:* Servidor MySQL/MariaDB para la persistencia relacional de históricos y configuraciones.

## Nivel de Cliente (Client Tier)

**Nodo:** Dispositivo del Usuario Final (Smartphone, Tablet o Laptop).

**Artefacto:** Navegador Web moderno compatible con HTML5/JavaScript para renderizar el M.I.P.

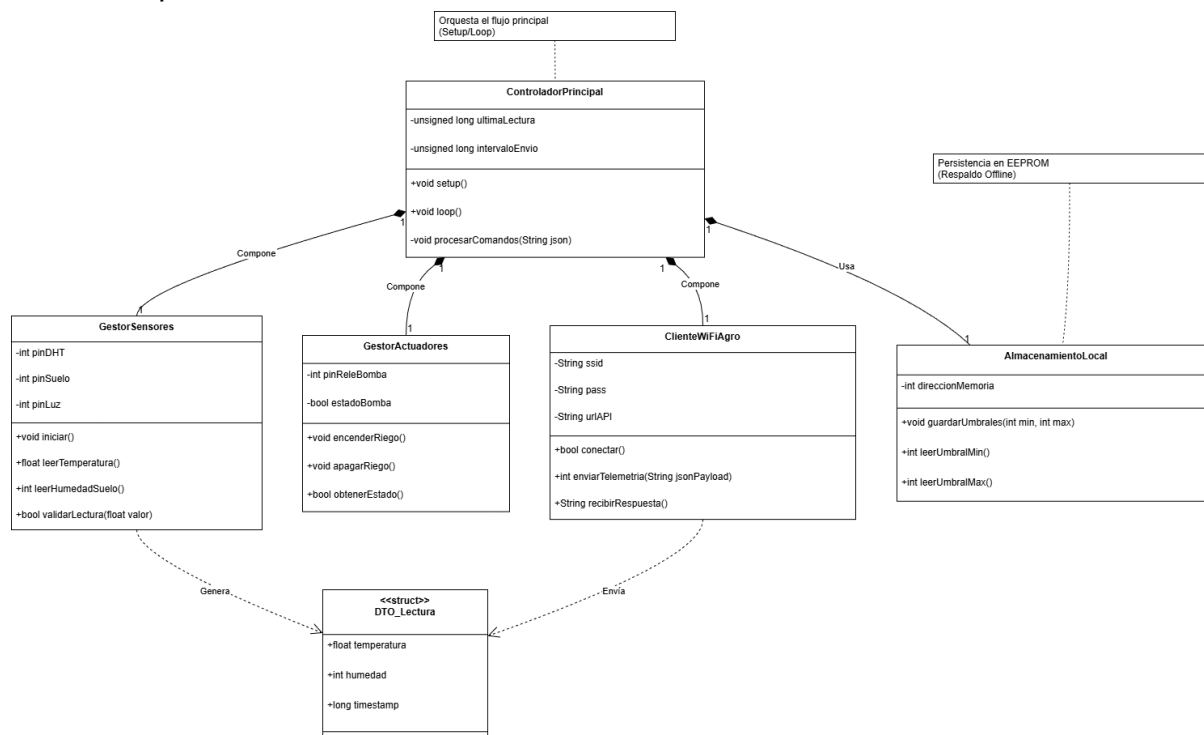
## 3. Diseño Detallado de Módulos (Diagramas de Clases)

Esta sección presenta la estructura estática del software mediante Diagramas de Clases UML. Cada diagrama detalla los atributos, métodos y relaciones de las clases que implementan la lógica de negocio descrita en la ERS, refinando el modelo de dominio inicial hacia una arquitectura orientada a objetos lista para la programación.



## 3.1. Diagrama de Clases: Módulo de Adquisición y Control (M.A.C.)

Realizado por David Alvarado.



Este diagrama representa la estructura interna del software embebido, diseñado para ejecutarse en el microcontrolador bajo restricciones de tiempo real y memoria limitada.

- **ControladorPrincipal:** Su responsabilidad es orquestar el ciclo de vida (setup/loop), gestionando los tiempos de sondeo mediante contadores no bloqueantes (millis) para asegurar que el dispositivo siempre responda a la red.
- **Abstracción de Hardware (HAL):** Se aplica el principio de responsabilidad única separando el acceso físico en dos gestores:
- **GestorSensores:** Encapsula la lectura de pines analógicos/digitales y la validación de datos (ej. descartar ruido o valores negativos).
- **GestorActuadores:** Controla los relés de potencia. Mantiene el estado lógico de la bomba (encendido/apagado) sincronizado con el estado físico.



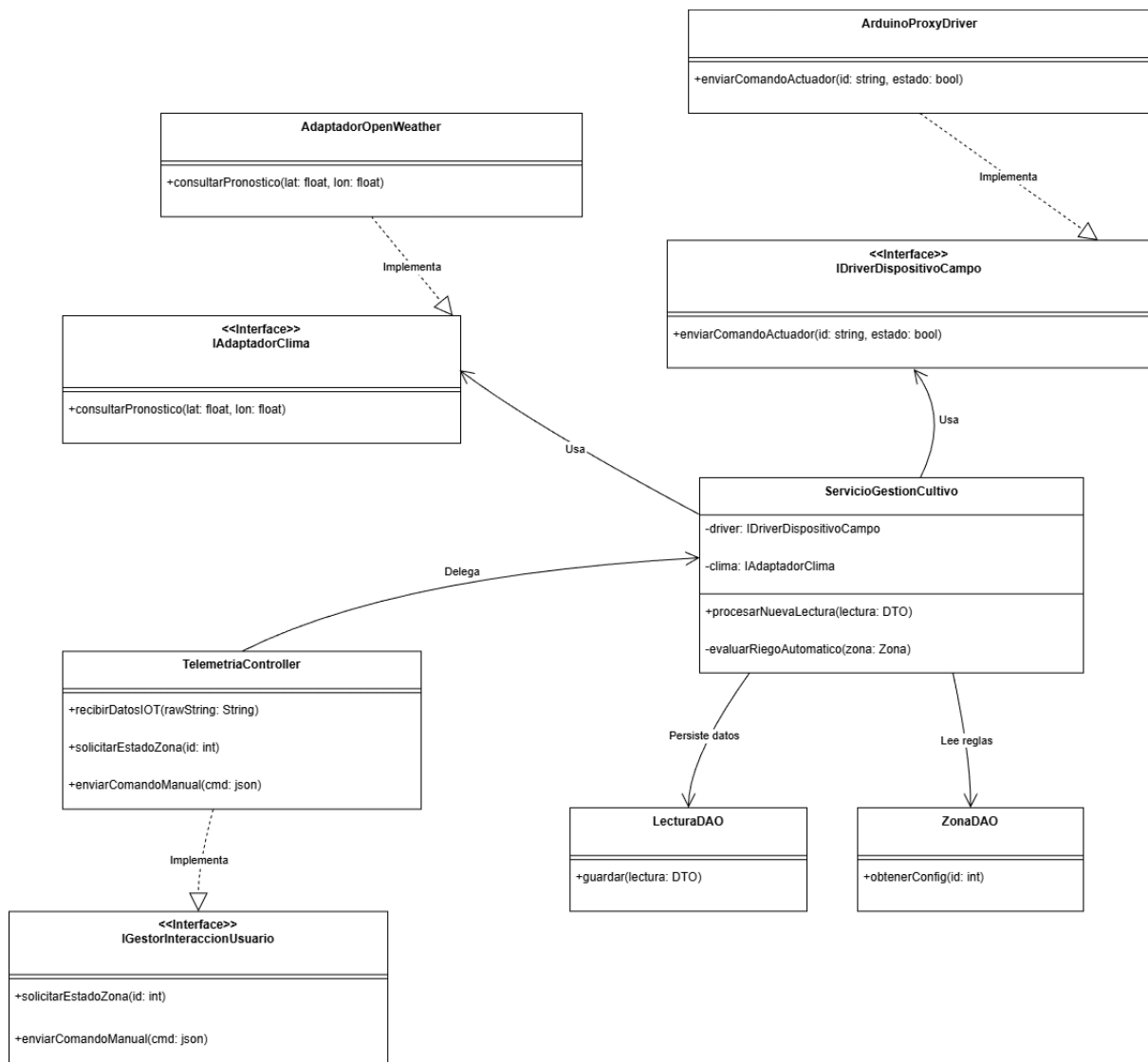
# UNIVERSIDAD DE CUENCA

- **ClienteWiFiAgro:** Esta clase encapsula la pila TCP/IP. Se ha diseñado para utilizar un protocolo de tramas de texto ligero (en lugar de objetos JSON pesados) en los métodos enviarTelemetry y recibirRespuesta, optimizando el uso de la memoria SRAM del dispositivo.
- **AlmacenamientoLocal:** Permite guardar la configuración crítica (umbrales de riego) en la memoria no volátil (EEPROM). Esto garantiza que, ante un reinicio inesperado o pérdida de conexión WiFi, el sistema pueda seguir operando con los últimos parámetros conocidos.



## 3.2. Diagrama de Clases: Módulo de Comunicaciones y Servidor (M.C.S.)

Realizado por Kevin Coraizaca. Backend / API REST (Node.js/Python + PostgreSQL).



Arquitectura de Clases del Servidor (Backend IoT):

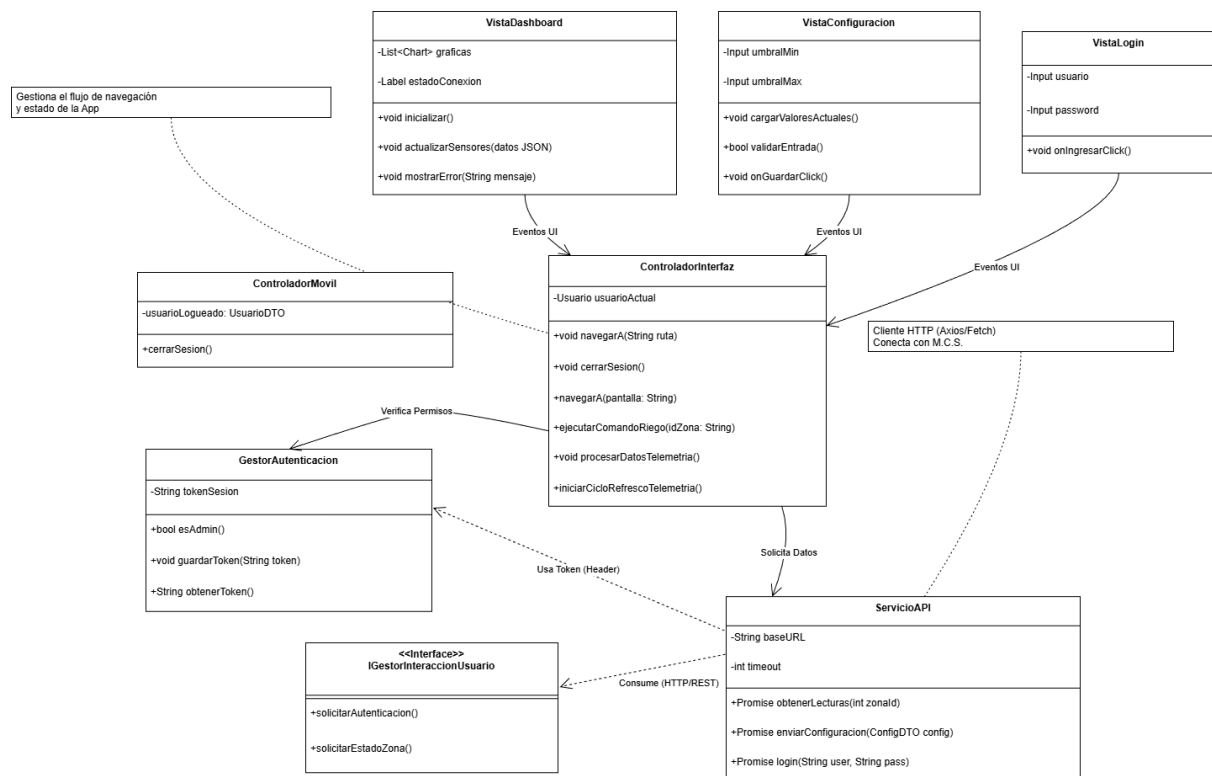
Diseñado bajo el principio de Inversión de Dependencias para garantizar un bajo acoplamiento entre la lógica de negocio y los dispositivos externos.



- **Capa de Controladores (TelemetriaController):** Actúa como el punto de entrada del sistema. Implementa explícitamente la interfaz `IGestorInteraccionUsuario`, garantizando que se cumple el contrato de comunicación que espera la aplicación móvil. Su función es delegar las peticiones a la capa de servicios, evitando contener lógica compleja.
- **Capa de Negocio (ServicioGestionCultivo):** Es el componente central ("Cerebro"). No depende de clases concretas, sino de las interfaces `IDriverDispositivoCampo` (para actuar sobre el hardware) y `IAdaptadorClima` (para consultar datos externos). Esto permite que el sistema decida, por ejemplo, suspender un riego si el pronóstico de clima indica lluvia, orquestando la información de múltiples fuentes.
- **Capa de Acceso a Datos (DAOs):** Las clases `LecturaDAO` y `ZonaDAO` abstraen la persistencia en la base de datos, permitiendo guardar historiales y configuraciones sin que el servicio conozca el lenguaje SQL.
- **Capa de Integración (Implementaciones):** Las clases `ArduinoProxyDriver` y `AdaptadorOpenWeather` son las encargadas de "ensuciarse las manos" con los detalles técnicos (protocolos, JSON, HTTP), encapsulando esta complejidad detrás de las interfaces definidas.

### 3.3. Diagrama de Clases: Módulo de Interfaz (M.I.P.)

Realizado por David Alvarado. Frontend (Aplicación Web / Móvil).



Se detalla la estructura interna de la aplicación de usuario, siguiendo un patrón MVC del lado del cliente para separar la gestión de la interfaz gráfica de la lógica de comunicación.

- **Capa de Vistas (UI)**: Componentes como **VistaDashboard** y **VistaConfiguracion** son responsables únicamente de la presentación. No procesan datos, sino que notifican eventos de usuario a la capa de control.
- **ControladorInterfaz**: Gestiona el estado de la sesión (**usuarioActual**) y la navegación de la aplicación. Actúa como intermediario, recibiendo eventos de las vistas y solicitando datos a la capa de servicios.
- **ServicioAPI**: Encapsula el cliente HTTP (Axios/Fetch). Integración Contractual: Este diseño es que la clase **ServicioAPI** consume la interfaz **IgestorInteraccionUsuario**.
- **GestorAutenticacion**: Un componente transversal encargado de almacenar y suministrar el Token de sesión (JWT) en cada petición, manteniendo la seguridad separada de la lógica funcional.



## 4. Diseño de Interfaces Gráficas de Usuario

Esta sección define la experiencia de usuario (UX) y la interfaz visual (UI) del sistema AgroSmart. El diseño prioriza la **usabilidad en campo**, optando por componentes de alto contraste y una arquitectura de información plana para facilitar el acceso rápido a la telemetría y configuración.

El diseño sigue un sistema de diseño (Design System) unificado basado en **Material Design**, utilizando una paleta de colores coherente (Verde Institucional #4CAF50 para estados seguros, Rojo #F44336 para alertas) y tipografía sans-serif para maximizar la legibilidad en dispositivos móviles.

A continuación, se presentan las interfaces diseñadas por cada integrante, vinculando explícitamente los campos visuales con los atributos definidos en los diagramas de clases de la Sección 3.

### 4.1. Interfaz de Monitoreo de Cultivo (Dashboard)

- **Estudiante Responsable:** David Alvarado
- **Caso de Uso Relacionado:** RU-MIP-01 "Monitorear Variables por Zona"
- **Módulos Relacionados:** M.I.P. (Frontend) y M.C.S. (Datos)

Panel principal operativo. Permite al Operador visualizar el estado actual del cultivo en tiempo real. La interfaz se conecta mediante polling (cada 10s) para refrescar los valores sin recargar la página.

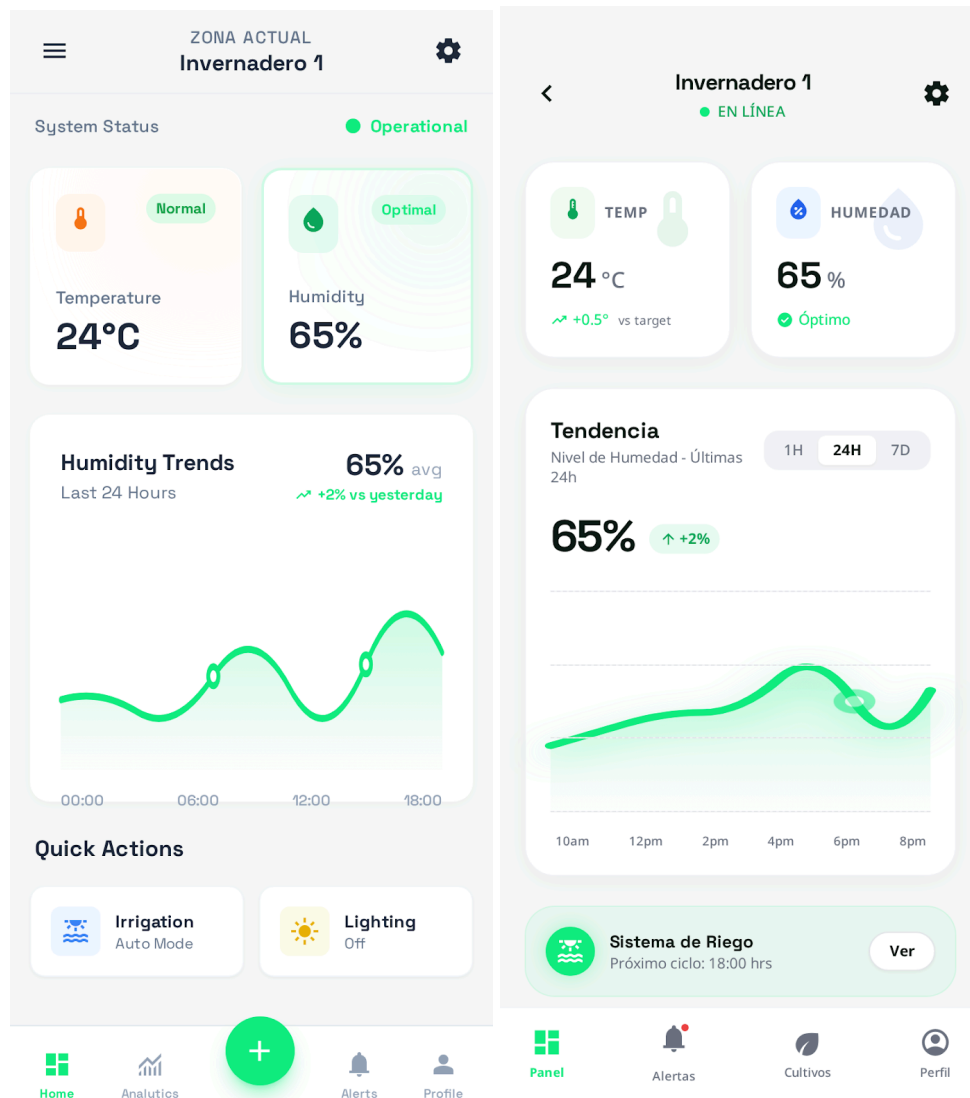
Mapeo de campos visuales a clases:

| Elemento Visual (GUI) | Clase Relacionada (Sec. 3) | Atributo Vinculado | Descripción Técnica                      |
|-----------------------|----------------------------|--------------------|------------------------------------------|
| Card: Temperatura     | LecturaDTO                 | float temperatura  | Valor numérico actual recibido del JSON. |
| Card: Humedad Suelo   | LecturaDTO                 | float humedad      | Valor porcentual (0-100%).               |
| Indicador Estado      | EntidadZona                | EstadoZona estado  | Icono dinámico (Check Verde /            |



# UNIVERSIDAD DE CUENCA

|                |                |                            |                                                 |
|----------------|----------------|----------------------------|-------------------------------------------------|
|                |                |                            | Alerta Roja).                                   |
| Gráfica Lineal | VistaDashboard | List<Lectura><br>historial | Array de objetos<br>renderizados en<br>eje X/Y. |





## 4.2. Interfaz de Configuración de Automatización (Reglas)

- **Estudiante Responsable:** Kevin Coraizaca
- **Caso de Uso:** RU-MGC-01 "Configurar Umbrales de Control"
- **Módulos Relacionados:** M.G.C. (Lógica de Negocio)

Formulario de gestión que permite definir las reglas lógicas para la activación automática del riego. Esta interfaz interactúa directamente con el motor de reglas del servidor, actualizando los parámetros que rigen el comportamiento autónomo.

Mapeo de campos visuales a clases:

| Elemento Visual (GUI) | Clase Relacionada (Sec. 3) | Atributo Vinculado      | Descripción Técnica                      |
|-----------------------|----------------------------|-------------------------|------------------------------------------|
| Slider: Mínimo        | EntidadZona                | float umbralHumedad Min | Valor de disparo de encendido (ej. 30%). |
| Slider: Máximo        | EntidadZona                | float umbralHumedad Max | Valor de disparo de apagado (ej. 80%).   |
| Toggle: Auto          | EntidadZona                | boolean modoAutomatico  | Switch para habilitar/deshabilitar       |
| Botón: Guardar        | TelemetriaControl          | configurarUmbrales()    | Disparador del método POST al backend.   |



## < Configuración de Riego



### Modo Automático

El sistema regará según los sensores



### Umbrales de Humedad

Mínimo (Encender)

30%



Máximo (Apagar)

80%



El riego se activará cuando la humedad del suelo baje del 30% y se detendrá al alcanzar el 80%.



GUARDAR CAMBIOS



## 4.3. Interfaz de Control Manual de Actuadores

- **Estudiante Responsable:** David Alvarado
- **Caso de Uso:** RU-MAC-03 "Control Manual de Riego"
- **Módulos Relacionados:** M.A.C. (Hardware)

Interfaz de mando directo destinada a situaciones de emergencia o mantenimiento. Permite anular el sistema automático y forzar el estado de los actuadores físicos (bombas/válvulas). Requiere retroalimentación visual inmediata del estado de conectividad del dispositivo.

Mapeo de campos visuales a clases:

| Elemento Visual (GUI) | Clase Relacionada (Sec. 3) | Atributo Vinculado   | Descripción Técnica                           |
|-----------------------|----------------------------|----------------------|-----------------------------------------------|
| Botón Power           | GestorActuadores           | bool estadoBomba     | Botón circular grande. Verde=ON, Gris=OFF.    |
| Badge: Conexión       | ClienteWiFiAgro            | bool conectado       | Etiqueta "En Línea" o "Desconectado".         |
| Log de Eventos        | MainController             | String ultimoComando | Texto pequeño: "Última acción: Riego Manual". |



# UNIVERSIDAD DE CUENCA

← Control Manual

Bomba 1 



APAGAR

● ESTADO: RIEGO ACTIVO

💧 45 PSI • ⚡ 12.4 A



ÚLTIMA ACCIÓN  
Manual Start 10:00 AM





## 5. Diseño Dinámico (Diagramas de Secuencia Detallados)

Esta sección presenta la realización dinámica de los casos de uso críticos. A diferencia de los diagramas de la fase de análisis (ERS), estos modelos detallan la interacción entre las instancias de las clases definidas en el Diseño Detallado (Sección 3), mostrando el flujo exacto de mensajes, parámetros y tipos de retorno.

### 5.1. Realización del Caso de Uso: "Monitoreo Remoto de Cultivo"

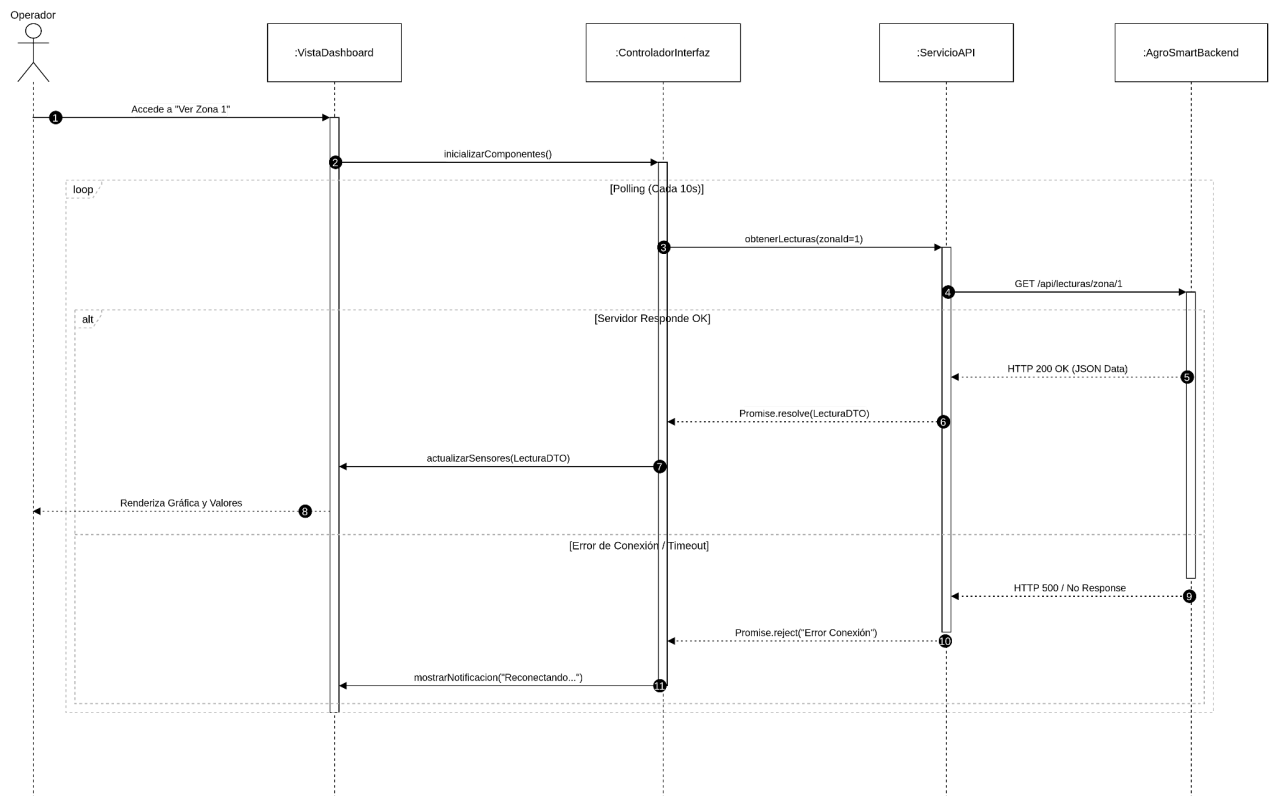
**Estudiante Responsable:** David Alvarado

**Módulo:** M.I.P. (Frontend)

**Caso de Uso:** RU-MIP-01

**Objetivo:** Mostrar cómo la interfaz solicita datos asíncronamente sin bloquear la experiencia de usuario.

El diagrama ilustra el patrón *Single Page Application*. La VistaDashboard delega la comunicación al ServicioAPI. Se utiliza un bloque alt para manejar el escenario donde el servidor no responde (Timeout), cumpliendo con el requisito no funcional de robustez.



## 5.2. Realización del Caso de Uso: "Procesamiento de Telemetría y Reglas"

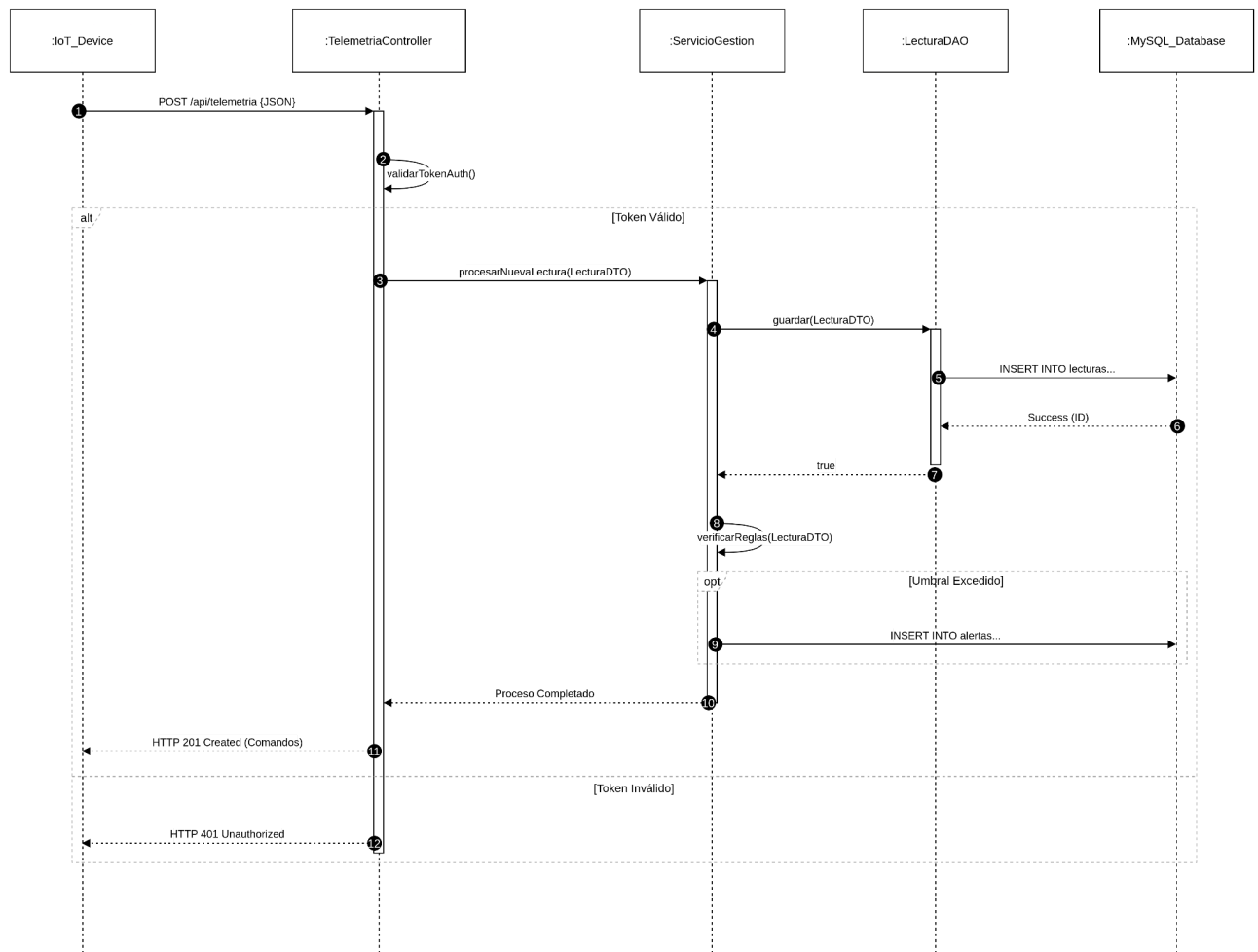
**Estudiante Responsable:** Kevin Coraizaca

**Módulo:** M.C.S. (Backend)

**Caso de Uso:** RU-MCS-01 (Persistencia) + RU-MGC-01 (Reglas)

**Objetivo:** Detallar el flujo interno del servidor cuando llega un dato, incluyendo la validación de negocio y el guardado en BD.

Este diagrama "Caja Blanca" muestra la arquitectura en capas del backend. El **TelemetriaController** recibe la petición, pero delega la lógica al **ServicioGestion**. Se destaca el uso del **LecturaDAO** para persistencia y la verificación automática de umbrales (M.G.C.) que podría disparar una alerta.



## 5.3. Realización del Caso de Uso: "Ciclo de Adquisición en Hardware"

**Estudiante Responsable:** David Alvarado

**Módulo:** M.A.C. (Hardware)

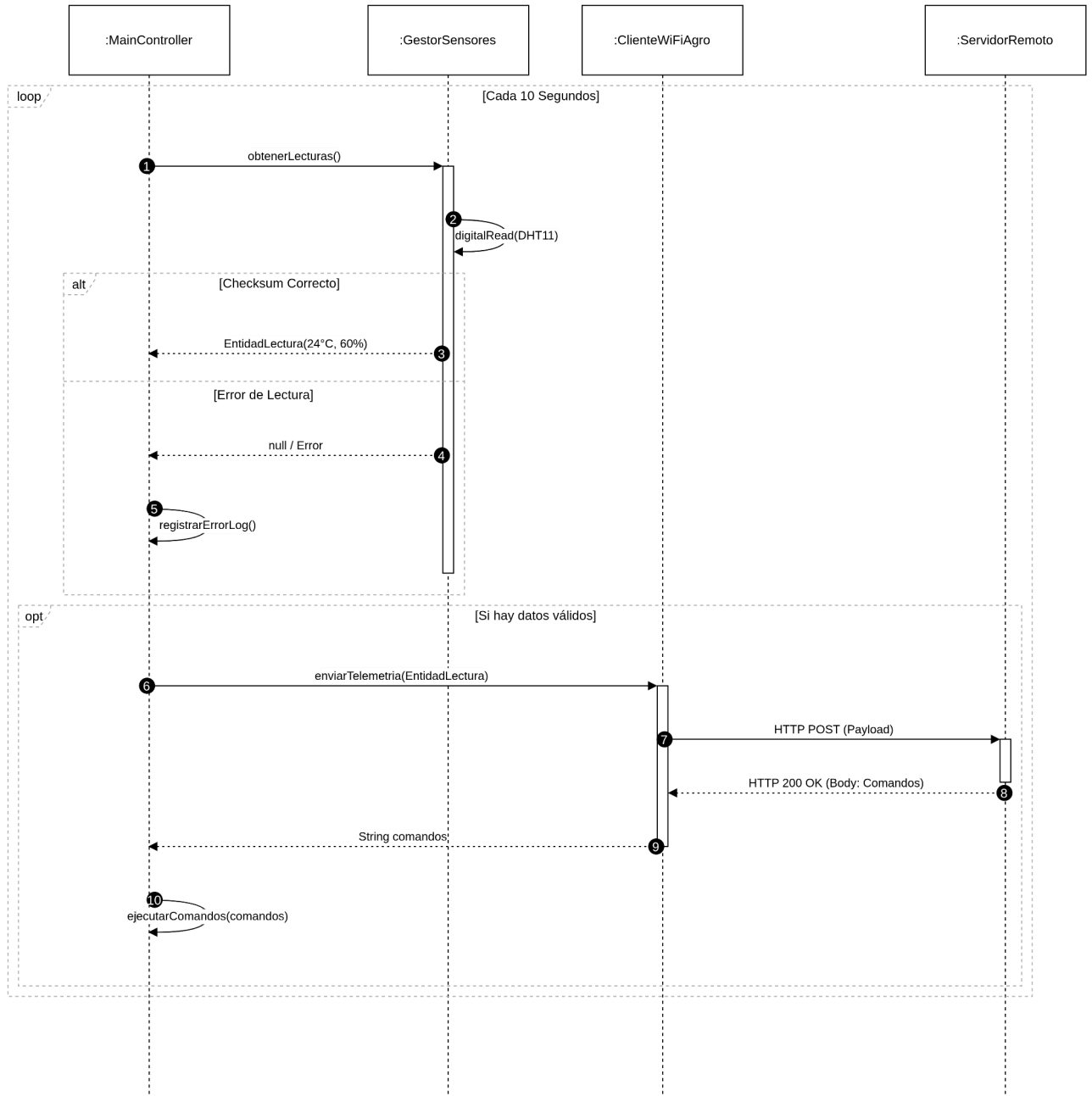
**Caso de Uso:** RU-MAC-01

**Objetivo:** Mostrar cómo el firmware controla el hardware físico y gestiona los tiempos de espera.

Representa el bucle principal (void loop) del microcontrolador. Muestra la interacción física con la clase GestorSensores y cómo el ClienteWiFi encapsula la complejidad de la red. Se incluye un fragmento opt para el manejo de fallos de lectura en el sensor físico.



# UNIVERSIDAD DE CUENCA



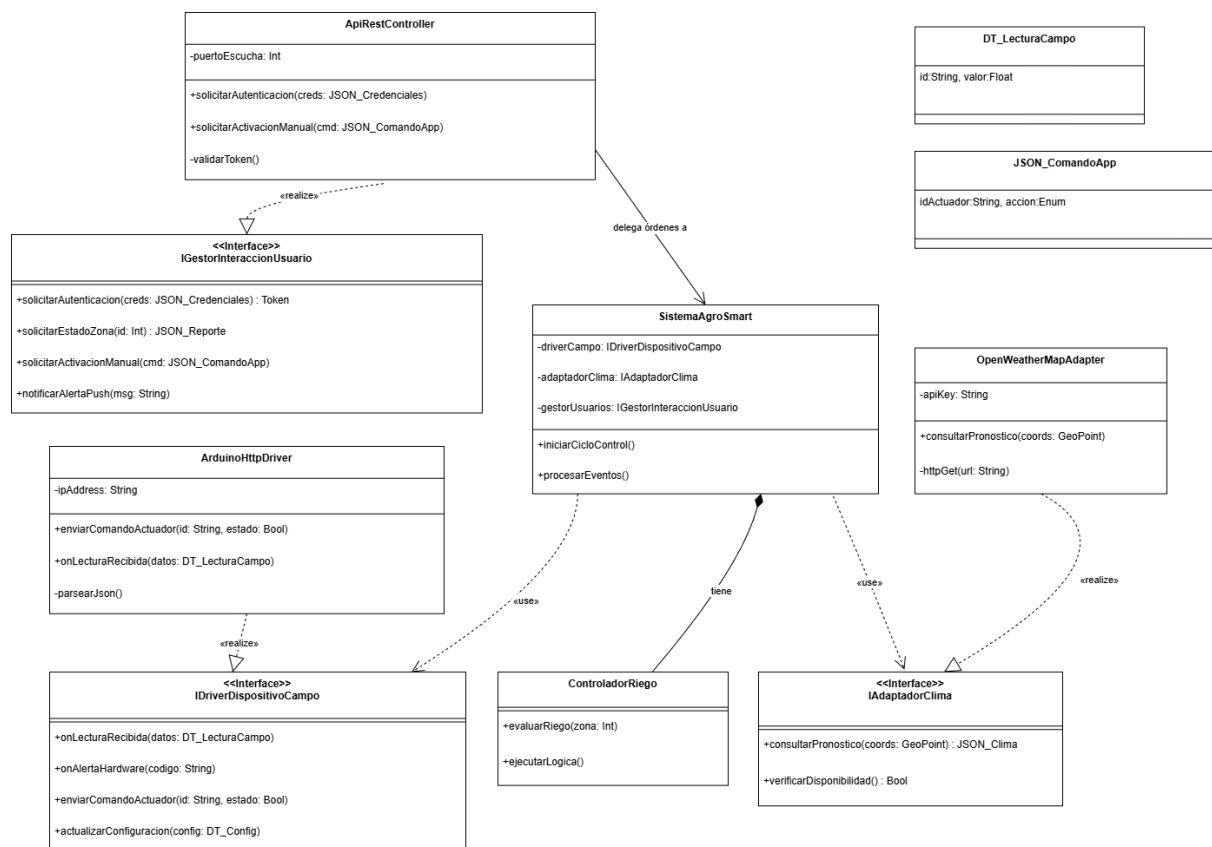


## 6. Diseño de Interfaz con Otros Sistemas

Esta sección especifica los contratos de comunicación entre el sistema AgroSmart y las entidades externas identificadas en el Modelo de Contexto de la ERS. Se detallan las interfaces lógicas (operaciones de software), físicas (conexiones y protocolos) y la estructura de datos intercambiada.

### Diagrama de Interfaces:

El siguiente diagrama de interfaces (clases) presenta la arquitectura de puertos y adaptadores del sistema.



**Hardware:** Se utiliza **IDriverDispositivoCampo** para aislar la comunicación con los sensores. La clase **ArduinoHttpDriver** es el adaptador concreto.

**Servicios Externos:** La interfaz **IAdaptadorClima** permite conectar con proveedores meteorológicos como **OpenWeatherMap** sin que la lógica central conozca la API específica.



**Usuario:** La interfaz IGestorInteraccionUsuario define el contrato de servicios que la aplicación móvil puede consumir. La clase ApiRestController implementa esta interfaz, exponiendo los endpoints REST que disparan la lógica interna del sistema.

**Definición Tipos de Datos:**

| Nombre             | Formato                    | Descripción / Estructura                                                      | Ejemplo                                                                                           |
|--------------------|----------------------------|-------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|
| DT_Lectura Campo   | Cadena Delimitada (Pipe  ) | Datos crudos de sensores.<br><br>Estructura:<br>ID_ZONA TEMP HUM LUZ RIEGO    | Z01 24.5 60 800 ON                                                                                |
| DT_Comando Control | Cadena Compacta            | Orden directa al hardware.<br><br>Estructura:<br>ACTUADOR_ID:E<br>STADO       | PUMP_01:1                                                                                         |
| JSON_Reporte Zona  | Objeto JSON                | Estado completo de una zona.<br><br>Estructura:<br>{componente1, componente2} | { "zona_id": "Z01",<br>"cultivo": "Tomate",<br>"temp": 24.5,<br>"humedad": 60,<br>"alertas": [] } |



|                       |                             |                                                                                        |                                                                                 |
|-----------------------|-----------------------------|----------------------------------------------------------------------------------------|---------------------------------------------------------------------------------|
| JSON_Config<br>Umbral | Objeto<br>JSON              | Reglas de<br>negocio.<br><br>Estructura:<br>{componente1,<br>componente2}              | { "cultivo_id": "C01",<br>"temp_min": 18.0,<br>"temp_max": 28.0 }               |
| JSON_Comand<br>oApp   | Objeto<br>JSON              | Acción manual<br>desde la App.<br><br>Estructura:<br>{componente1,<br>componente2}     | { "actuador_id":<br>"VENT_02", "accion":<br>"ENCENDER",<br>"usuario": "admin" } |
| JSON_Clima<br>Data    | Objeto<br>JSON<br>(Externo) | Respuesta de API<br>Meteorológica.<br><br>Estructura:<br>{componente1,<br>componente2} | { "forecast": "rain",<br>"precip_prob": 80,<br>"temp_max": 22 }                 |

## Interfaces:

### 1. Interfaz: IDriverDispositivoCampo (Proxy del Arduino)

Actor: Dispositivo Hardware (M.A.C.)

Contrato que estandariza la comunicación con los sensores y actuadores físicos.

| Nombre Método | Tipo | Descripción<br>Funcional | Firma / Parámetros |
|---------------|------|--------------------------|--------------------|
|               |      |                          |                    |



# UNIVERSIDAD DE CUENCA

|                           |          |                                                                                                                |                                                         |
|---------------------------|----------|----------------------------------------------------------------------------------------------------------------|---------------------------------------------------------|
| onLecturaRecibida         | INBOUND  | El driver reporta que llegaron nuevos datos de temperatura, humedad, etc. Se usa para guardar historial.       | onLecturaRecibida(datos: DT_LecturaCampo)               |
| onAlertaHardware          | INBOUND  | El dispositivo avisa de un fallo crítico (ej. sensor desconectado o nivel de batería bajo).                    | onAlertaHardware(codigo: String, nivel: Enum)           |
| onSolicitudSincronizacion | INBOUND  | El dispositivo solicita la hora actual o configuración al reiniciarse. (Equivale al antiguo sincronizarReloj). | onSolicitudSincronizacion(idDispositivo: String)        |
| onConfirmacionEjecucion   | INBOUND  | El dispositivo confirma que ejecutó una orden previa (ej. "Ya encendí la bomba").                              | onConfirmacionEjecucion(idOrden: Int, exito: Bool)      |
| enviarComandoActuador     | OUTBOUND | El sistema ordena cambiar el estado de un actuador físico (Riego, Ventilador, Luces).                          | enviarComandoActuador(idActuador: String, estado: Bool) |
| actualizarConfiguracion   | OUTBOUND | El sistema envía nuevos parámetros operativos al firmware (ej. cambiar                                         | actualizarConfiguracion(config: DT_ConfigHardware)      |



|                      |          |                                                      |                                             |
|----------------------|----------|------------------------------------------------------|---------------------------------------------|
|                      |          | frecuencia de sondeo).                               |                                             |
| reiniciarDispositivo | OUTBOUND | Orden forzada de reinicio remoto en caso de bloqueo. | reiniciarDispositivo(idDispositivo: String) |

## Especificación de Operaciones (Interfaz 1)

| Operación / Método    | Precondiciones                                                                                    | Postcondiciones                                                                           | Excepciones Posibles                                                                                                                                      | Protocolo   |
|-----------------------|---------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------|-------------|
| onLecturaRecibida     | El formato del mensaje recibido debe coincidir con la estructura `ID`                             |                                                                                           |                                                                                                                                                           | L`.         |
| enviarComandoActuador | El dispositivo debe estar en estado ONLINE y el ID del actuador debe existir en la configuración. | El actuador físico cambia de estado (ON/OFF) y el dispositivo retorna confirmación (ACK). | <p>tiempo_espera_agotado_exception (Timeout): El dispositivo no respondió en 500ms.</p> <p>dispositivo_desconectado_exception: No hay ping con la IP.</p> | HTTP (POST) |



# UNIVERSIDAD DE CUENCA

|                  |                          |                                                                     |                                     |            |
|------------------|--------------------------|---------------------------------------------------------------------|-------------------------------------|------------|
| sincronizarReloj | Conexión de red estable. | El reloj interno del Arduino se actualiza con la hora del servidor. | error_sincronizacion_ntp_exception. | NTP / HTTP |
|------------------|--------------------------|---------------------------------------------------------------------|-------------------------------------|------------|

## 2. Interfaz: IGestorInteraccionUsuario (App Móvil)

Actor: Usuario Humano (Admin/Operador) vía App (M.I.P.)

Contrato para todas las acciones que un humano puede realizar o recibir.

| Nombre Método          | Tipo    | Descripción Funcional                                                          | Firma / Parámetros                                      |
|------------------------|---------|--------------------------------------------------------------------------------|---------------------------------------------------------|
| solicitarAutenticacion | INBOUND | El usuario envía credenciales para obtener acceso. (Equivale a iniciarSesion). | solicitarAutenticacion(creds: JSON_Credenciales): Token |
| solicitarEstadoZona    | INBOUND | El usuario pide ver los datos actuales del dashboard.                          | solicitarEstadoZona(idZona: Int): JSON_ReporteZona      |



# UNIVERSIDAD DE CUENCA

|                           |          |                                                                                           |                                                            |
|---------------------------|----------|-------------------------------------------------------------------------------------------|------------------------------------------------------------|
| solicitarHistorial        | INBOUND  | El usuario pide un rango de datos pasados para gráficas.                                  | solicitarHistorial(filtro: JSON_RangoFechas): Lista<Datos> |
| solicitarActivacionManual | INBOUND  | El usuario presiona un botón para encender/apagar algo manualmente.                       | solicitarActivacionManual(comando: JSON_ComandoApp)        |
| solicitarCambioUmbrales   | INBOUND  | El usuario (Admin) envía nuevas reglas de temperatura/humedad.                            | solicitarCambioUmbrales(reglas: JSON_ConfigUmbral)         |
| notificarAlertaPush       | OUTBOUND | El sistema envía proactivamente un aviso al celular del usuario (ej. "Lluvia detectada"). | notificarAlertaPush(titulo: String, mensaje: String)       |
| enviarReporteMensual      | OUTBOUND | El sistema genera y envía un PDF/Resumen al correo o App del usuario.                     | enviarReporteMensual(usuarioId: String, reporte: Blob)     |



## Especificación de Operaciones (Interfaz 2)

| Operación / Método          | Precondiciones                                                                        | Postcondiciones                                                                   | Excepciones Posibles                                                                                                                               | Protocolo         |
|-----------------------------|---------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|-------------------|
| consultarPr<br>onostico     | La API Key debe ser válida y tener cuota disponible. Coordenadas geográficas válidas. | Retorna un objeto JSON_Clima con la probabilidad de lluvia para las próximas 24h. | api_no_autorizada_exception: Clave API expirada o inválida (HTTP 401).<br><br>servicio_no_disponible_exception: Servidor externo caído (HTTP 503). | HTTPS (REST/JSON) |
| verificarDis<br>ponibilidad | Conexión a Internet activa en el servidor.                                            | Retorna True si el servicio responde al Ping en < 200ms.                          | timeout_conexion_exception.                                                                                                                        | ICMP / HTTPS      |

### 3. Interfaz: IAdaptadorClima (Servicio Externo)

Actor: API Meteorológica (M.I.E.)

Contrato para obtener información del entorno externo.

| Nombre Método | Tipo | Descripción Funcional | Firma / Parámetros |
|---------------|------|-----------------------|--------------------|
|---------------|------|-----------------------|--------------------|



|                         |          |                                                              |                                                       |
|-------------------------|----------|--------------------------------------------------------------|-------------------------------------------------------|
| consultarPronostico     | OUTBOUND | El sistema pregunta si va a llover en las próximas horas.    | consultarPronostico(coords: GeoPoint): JSON_ClimaData |
| verificarDisponibilidad | OUTBOUND | Ping para saber si el servicio externo de clima está online. | verificarDisponibilidad(): Bool                       |

## 7. Matriz de Trazabilidad

Esta sección presenta las matrices de rastreo que garantizan la cobertura total de los requerimientos definidos en la ERS. Para facilitar la auditoría del diseño, se han elaborado tres matrices con enfoques complementarios: Estructural, de Interfaz y Físico.

### 7.1. Matriz de Trazabilidad: Requerimientos vs. Clases de Software

- **Estudiante Responsable:** Kevin Coraizaca
- **Enfoque:** Estructural (Backend y Lógica)
- **Descripción:** Esta matriz vincula los Requerimientos Funcionales (RU) con las **Clases y Componentes del Diagrama de Diseño (Sección 3)** que implementan dicha lógica. Permite verificar que cada regla de negocio tiene código asociado.

| ID Requerimiento | Descripción Corta     | Módulo | Clase / Componente de Diseño | Método Clave / Responsabilidad  |
|------------------|-----------------------|--------|------------------------------|---------------------------------|
| RU-MCS-01        | Persistencia de Datos | M.C.S. | LecturaDAO                   | guardar(): Inserción SQL en BD. |



|              |                           |        |                        |                                          |
|--------------|---------------------------|--------|------------------------|------------------------------------------|
| RU-MCS-03    | Verificar Conectividad    | M.C.S  | TelemetryController    | checkStatus(): Validación de heartbeats. |
| RU-MGC-01    | Configurar Umbrales       | M.G.C. | ServicioGestionCultivo | actualizarReglas(): Lógica de negocio.   |
| RU-MIE-01    | Pronóstico del Clima      | M.I.E. | AdaptadorOpenWeather   | consultarApi(): Consumo REST externo.    |
| RS-MCS-01.E1 | Validación de Tramas      | M.C.S  | TelemetryController    | validarSchema(): Integridad de datos.    |
| RNF-P-03     | Rendimiento Transaccional | M.C.S  | PoolConexiones (DAO)   | Gestión eficiente de hilos de BD.        |

## 7.2. Matriz de Trazabilidad: Casos de Uso vs. Interfaces Gráficas

- **Estudiante Responsable:** David Alvarado
- **Enfoque:** Interacción (Frontend y UX)
- **Descripción:** Esta matriz relaciona los Casos de Uso del Usuario con las **Pantallas (Vistas)** diseñadas en la Sección 4. Su objetivo es asegurar que el usuario tenga un medio visual para ejecutar cada función prometida.

| ID Caso de Uso | Nombre del Caso de Uso | Actor Principal | Interfaz Gráfica (GUI) | Elemento Visual Clave                   |
|----------------|------------------------|-----------------|------------------------|-----------------------------------------|
| RU-M IP-01     | Monitorear Variables   | Operador        | VistaDashboard         | Gráficas de Línea y Tarjetas de Sensor. |
| RU-M IP-04     | Registro Mantenimiento | Técnico         | VistaMantenimiento     | Formulario de Bitácora.                 |



|                   |                          |          |                    |                                            |
|-------------------|--------------------------|----------|--------------------|--------------------------------------------|
| RU-M<br>GC-0<br>1 | Configurar<br>Parámetros | Admin    | VistaConfiguracion | Sliders de<br>Umbrales<br>(Min/Max).       |
| RNF-<br>S-01      | Autenticación            | Todos    | VistaLogin         | Campos<br>User/Pass<br>y Botón<br>Ingreso. |
| RU-M<br>AC-03     | Control Manual<br>Riego  | Operador | VistaDetalleZona   | Botón<br>Toggle<br>(ON/OFF)<br>Riego.      |

### 7.3. Matriz de Trazabilidad: Requerimientos vs. Hardware y Firmware

- **Estudiante Responsable:** Kevin Coraizaca
- **Enfoque:** Físico (Hardware y Señales)
- **Descripción:** Esta matriz mapea los requerimientos de adquisición y control con los **Dispositivos Físicos** y las funciones específicas del **Firmware** (Sección 2.3 y 3.1). Garantiza que el hardware seleccionado soporta las necesidades del software.

| ID<br>Requerimie<br>nto | Variable /<br>Acción      | Compone<br>nte Físico<br>(Hardwar<br>e) | Función Firmware (C++)            | Pin Físico<br>(Keyestu<br>dio) |
|-------------------------|---------------------------|-----------------------------------------|-----------------------------------|--------------------------------|
| RU-MAC-0<br>1           | Leer<br>Temperat<br>ura   | Sensor<br>DHT11                         | GestorSensores::leerTem<br>p()    | D2<br>(Digital)                |
| RU-MAC-0<br>1           | Leer<br>Humeda<br>d Suelo | Higrómetr<br>o<br>Capacitiv<br>o        | GestorSensores::leerHum<br>edad() | A0<br>(Analógic<br>o)          |
| RU-MAC-0<br>3           | Activar<br>Riego          | Módulo<br>Relé 5V                       | GestorActuadores::activar<br>( )  | D7<br>(Digital<br>Out)         |
| RS-MAC-0<br>1.1         | Transmis<br>ión WiFi      | Módulo<br>ESP8266                       | ClienteWiFi::post()               | TX/RX<br>(Serial)              |



|              |                      |                         |                           |               |
|--------------|----------------------|-------------------------|---------------------------|---------------|
| RS-MAC-01.E3 | Respaldo Offline     | Memoria EEPROM (ATmega) | Almacenamiento::guardar() | N/A (Interno) |
| RU-MAC-04    | Mostrar Estado Local | Pantalla LCD I2C        | DisplayMgr::printStatus() | SDA/SCL (I2C) |

## 8. Referencias

- **Diagramas dibujados (abrir con Draw IO):**  
<https://drive.google.com/file/d/1VGmN-J9YtNWYFrLCCoeOdfthVtO9aefx/view?usp=sharing>
- **Documentación del dispositivo:**  
[https://wiki.keyestudio.com/KS0085\\_Keyestudio\\_Smart\\_Home\\_Kit\\_for\\_Arduino](https://wiki.keyestudio.com/KS0085_Keyestudio_Smart_Home_Kit_for_Arduino)
- **Arduino Docs.** "Uso de Arduino para agricultura". [En línea]. Disponible: <https://docs.arduino.cc>. [Consultado: 10-Nov-2025].
- **Adafruit.** "Sensores DHT11 especificaciones técnicas". [En línea]. Disponible: <https://learn.adafruit.com/dht>. [Consultado: 08-Nov-2025].
- **OMG.** "Unified Modeling Language (UML) Specification Version 2.5". Object Management Group. [En línea]. Disponible: <https://uml.org>.
- **Espressif Systems.** "ESP32 HTTP Client Protocol". [En línea]. Disponible: <https://docs.espressif.com>.
- **OpenWeatherMap.** "Current Weather Data API Documentation". [En línea]. Disponible: <https://openweathermap.org/api>.
- **IEEE Std 1016-2009.** "IEEE Standard for Information Technology--Systems Design--Software Design Descriptions". IEEE Computer Society.